AgentShield-Crypto: Evaluating Cryptographic Authorization, Durable Replay Protection, and Effect Safety for Tool-Using AI Agents

Saiful Islam Tanvir
AI and Cybersecurity Researcher

***Abstract*—Tool-using AI agents can turn a language-model proposal into an external action, such as placing an order or invoking a privileged service. The security problem is not only whether the proposal is syntactically valid: a system must also establish who authorized it, which policy and resource it applies to, whether the authorization is still valid, and whether retries or concurrent workers can produce duplicate effects. This paper presents AGENTSHIELD-CRYPTO, a research prototype for evaluating these controls in a single action-authorization path. The prototype combines strict message parsing, cryptographic signatures, key lifecycle checks, policy enforcement, expiry and audience validation, request-to-proposal binding, a durable SQLite replay ledger, and an idempotent simulated exchange. Its evaluation compares nine ablated designs across 17 attack or residual-risk families and a valid-request control, then examines crash points, concurrent submissions, revocation-cache staleness, and end-to-end latency. In the recorded full run, the full design executed zero of 1,020 attack requests while accepting all 60 valid controls; an intentionally weak algorithm-selection variant executed 60 attacks from the verifier-key-leak family. In 30 trials with eight concurrent workers, the durable design recorded exactly one effect per trial, whereas the process-local replay design recorded eight. Crash experiments also show why exactly-once behavior depends on idempotent downstream operations or reconciliation rather than on a local ledger alone. These results support a bounded engineering conclusion: layered authorization can prevent the tested request-level attacks, but it does not determine whether an in-policy action is semantically safe, and it cannot guarantee exactly-once effects against an arbitrary non-idempotent external system. The experiments are synthetic and do not establish production security or universal guarantees.**

***Index Terms*—AI agents, cybersecurity, authorization, digital signatures, replay attacks, idempotency, durable ledger, tool use, security evaluation.**

I. INTRODUCTION

AI agents increasingly operate through tools rather than producing text alone. A model may recommend a purchase, update a record, execute a deployment, or call an API. In such systems, a plausible-looking model output can become a consequential side effect. This changes the security boundary: the application must decide not simply whether a model response looks reasonable, but whether a particular action is authorized and safe to execute.

A common implementation mistake is to treat a tool call as a trusted continuation of the model conversation. Yet the proposal may be malformed, altered after approval, replayed after a restart, issued under a revoked key, or valid at the protocol level while violating business policy. Concurrent workers and retries introduce a second class of problem. Even if a request is authorized only once in principle, an ambiguous timeout can leave the caller unsure whether the external operation already occurred. Retrying without a durable idempotency mechanism may duplicate the effect.

This paper studies these problems through AGENTSHIELD-CRYPTO, a Python prototype that places a deterministic authorization gate between an agent proposal and a simulated exchange. The gate is designed to make several decisions explicit: parse the wire message strictly; verify its cryptographic envelope; validate key status and mandate lifetime; enforce policy; bind the signed mandate to the exact proposal; reserve the request identifier durably; and dispatch the effect through an idempotent exchange. The implementation uses Ed25519 for the full design and includes HMAC and intentionally weaker variants for controlled comparisons. The prototype uses SQLite to make the persistence and transaction boundaries inspectable.

The central research question is deliberately narrow: *How much protection do layered cryptographic authorization and durable effect-handling controls provide against a defined set of request-level attacks, and what residual risks remain?* The experiments are not intended to establish that a language model can reliably identify harmful intent, nor that the prototype is ready for deployment in a financial or other safety-critical environment.

The notebook evaluates a sequence of ablations, from no checks to a full design. The full design is tested against forged or malformed envelopes, signature changes, proposal substitution, replay, invalid key states, expired mandates, audience mismatch, policy-version downgrade, signed policy violations, and algorithm-selection confusion. It also includes a deliberately in-policy semantic manipulation family. This last family is important: a cryptographically valid request can still be unsafe if the policy or approval process permits the wrong action.

The main contributions of this work are:

- An executable reference design that separates signed authorization from the untrusted proposal and binds the two using a proposal digest.
- A controlled ablation study over nine designs and 1,020 attack requests per design, with valid controls and an explicitly retained semantic residual-risk family.
- A crash-point and concurrency evaluation that distinguishes request deduplication from downstream effect idempotency.

- An empirical examination of the performance impact of SQLite journaling and synchronization settings, together with a revocation-cache staleness experiment.

These contributions are implementation and evaluation contributions. We do not claim a new signature primitive, a general solution to prompt injection, or a universal exactly-once protocol. The manuscript should be read as a reproducible prototype study whose findings are bounded by the tested model, environment, and workload.

II. Problem Statement and Threat Model

A. System model

We consider a tool-using agent that produces a proposal p and a separate authorization service that issues a signed mandate m. A gate receives the wire envelope w, the proposal p, the current time t, and access to a key registry, policy configuration, durable request ledger, and downstream exchange. The exchange is the component that creates the externally observable effect.

The intended flow is:

$$\text{agent proposal} \rightarrow \text{authorization} \rightarrow \text{signed mandate} \rightarrow \mathsf{Gate} \rightarrow \mathsf{Exchange}.$$

The model output is not itself a credential. The gate treats the proposal and wire envelope as untrusted inputs and accepts only when the required checks succeed.

A proposal in the prototype contains a token, an action, and an integer amount in cents. The mandate contains a request identifier, audience, policy version, issue and expiry times, a digest of the proposal, and a decision with an approval bit and action fields. The exact schema is intentionally small enough to inspect and test.

B. Adversary capabilities

The test adversary can submit arbitrary bytes, malformed JSON, unknown key identifiers, altered signatures, altered proposals, stale or replayed requests, expired mandates, and signed requests that violate policy. It can also exploit an intentionally weak design that trusts the algorithm label in the incoming envelope. The attacker may submit requests concurrently through multiple processes and may retry after simulated failures.

The adversary is not assumed to possess an uncompromised signing key for the full design. The experiments do include a verifier-material leakage scenario to illustrate the difference between a public verification key and a shared HMAC secret. The prototype uses ephemeral test keys; these are not production key-management infrastructure.

C. Trust assumptions

The prototype assumes that the configured policy and trusted key registry are managed by a trusted operator, that the signing key is not compromised in the main full-design experiments, and that the gate's local SQLite files are available and correctly configured. It also assumes the simulated exchange implements the stated idempotency behavior when that option is enabled. These assumptions are essential to interpreting the results.

The prototype does not model a malicious operating system, compromised database administrator, rollback of the entire storage volume, Byzantine replicas, cross-region partitions, compromised authorization service, or a production key-management service. It does not establish that a policy is semantically correct. These are outside the present evaluation.

D. Security objectives

We evaluate the following objectives:

O1: Envelope integrity.
Altered or unauthenticated messages should not pass signature verification.

O2: Context validity.
A mandate must match the audience, policy version, key status, and time window.

O3: Proposal binding.
The submitted proposal must match the proposal authorized by the mandate.

O4: Replay resistance.
A request identifier must not produce a second accepted dispatch after it has been consumed, including across process restarts.

O5: Effect deduplication.
Concurrent requests and retries should not create duplicate exchange effects under the stated downstream idempotency assumptions.

O6: Explicit residual risk.
A request that is cryptographically valid and in policy may still encode a harmful or unintended goal; the gate is not a general semantic safety oracle.

III. Related Work and Context

A. Cryptographic message integrity

Digital signatures and message authentication codes are established mechanisms for message integrity and origin authentication. Ed25519 provides a public-key signature interface, while HMAC is a symmetric-key message authentication construction [1], [2]. This work does not propose a new primitive. Its focus is how cryptographic checks interact with policy, freshness, request binding, persistent replay state, and effect handling in one tool-execution path.

B. Canonicalization and protocol interpretation

Signed structured data must have an unambiguous representation. JSON permits formatting variation and, in common parser implementations, duplicate member names can be interpreted inconsistently. RFC 8785 defines a canonicalization scheme for JSON [3]. The prototype rejects duplicate keys and uses a restricted serializer; a production implementation should either adopt a standard profile or publish cross-language test vectors. This is a protocol-engineering requirement, not a novel cryptographic contribution.

C. Durability, retries, and distributed effects

Distributed-systems literature emphasizes that local state and remote side effects do not automatically form one atomic transaction. Recovery after an uncertain response therefore requires an explicit idempotency or reconciliation contract; a local reservation alone cannot determine whether an independent remote service applied an operation. General distributed-system design texts discuss these failure boundaries and the assumptions required for stronger guarantees [5], [6]. Our evaluation narrows this issue to a local SQLite ledger and a simulated exchange, and should not be interpreted as a new distributed commit protocol.

D. Agent security context

Security guidance for large language model applications increasingly treats tool invocation, excessive agency, and trust boundaries as application-level concerns rather than relying only on model behavior [8]. The present prototype addresses a specific part of that boundary: validating an authorization artifact and controlling repeated effects. It does not evaluate the full range of prompt injection, data exfiltration, model alignment, or multi-agent threats. A complete submission should expand this discussion with a verified, current comparison to peer-reviewed agent-security systems and explain the precise difference between those systems and this prototype.

IV. Design

A. Strict message parsing and canonical representation

The parser rejects oversized messages, duplicate JSON keys, floating-point values, and non-finite constants. Duplicate-key rejection matters because parsers can disagree about whether the first or last value wins. The prototype serializes structured data using sorted keys, compact separators, ASCII escaping, and disallows non-finite numbers. Canonicalization is a recognized concern in signed JSON protocols; RFC 8785 specifies a JSON Canonicalization Scheme, although the present implementation uses its own deliberately restricted serialization and should not be described as JCS-compliant without a conformance test [3]. The signed input includes a domain-separation prefix and the version, algorithm identifier, key identifier, and mandate.

Let $C(x)$ denote the prototype's canonical serialization and let D be the fixed domain-separation string. The signing input is

$$s_{\mathrm{in}} = D \,\|\, C(\{v, alg, kid, m\}).$$

The signature does not cover an arbitrary re-serialization chosen by a receiver; it covers the canonical representation produced by the protocol. This is a prototype-specific canonicalization scheme, not a claim of full compliance with a general JSON canonicalization standard. A production implementation should select and document a standard canonicalization profile and test cross-language interoperability.

B. Cryptographic envelope and algorithm pinning

The full design uses Ed25519 signatures, standardized in RFC 8032 [1], and looks up the verification key through a trusted registry. The verifier checks the algorithm against the configured key record rather than treating an untrusted envelope field as authoritative. The ablation `D7n naive alg` intentionally weakens this behavior to test algorithm-selection confusion.

HMAC-SHA-256, specified in RFC 2104 [2], is included in earlier designs as a comparison. HMAC verification requires the verifier to possess the same secret used to sign messages; exposure of that secret can therefore enable forgery. By contrast, an Ed25519 verifier uses public key material, which does not by itself enable signature creation. The experiment's verifier-leak family captures this distinction within the synthetic setup; it should not be interpreted as a comprehensive key-compromise study.

C. Mandate validation

The gate validates the envelope and mandate in a fixed sequence. In simplified form, acceptance requires:

$$\mathsf{Verify}_K(s_{\mathrm{in}}, \sigma) = 1, \tag{1}$$

$$t_{\mathrm{nb}}(K) \le t \le t_{\mathrm{na}}(K), \tag{2}$$

$$t_{\mathrm{issued}} - \delta \le t \le t_{\mathrm{expiry}} + \delta, \tag{3}$$

$$audience(m) = audience_{\mathrm{local}}, \tag{4}$$

$$version(m) = version_{\mathrm{policy}}, \tag{5}$$

$$\mathsf{H}(C(p)) = proposalHash(m), \tag{6}$$

$$PolicyAllows(p) = 1. \tag{7}$$

Here δ is the configured clock-skew allowance. The prototype uses an explicit maximum mandate lifetime and bounded wire size. Key lifecycle checks reject unknown, not-yet-valid, expired, or revoked keys. The registry can cache lookups; the cache experiment measures the resulting bounded stale-acceptance window after revocation.

D. Durable replay ledger

A process-local set is insufficient when several workers maintain independent memory. The prototype therefore reserves request identifiers in SQLite using a transaction that acquires a write lock before inserting. A uniqueness constraint on the request identifier provides the cross-process exclusion point. The ledger stores a request fingerprint and a state such as `RESERVED`, `DISPATCHING`, or `CONFIRMED`. Reuse of an identifier with a different fingerprint is treated as a conflict rather than as a legitimate retry.

The reservation prevents multiple workers from independently claiming the same request. However, a ledger reservation alone cannot atomically commit an effect in an unrelated external system. This is the classic boundary between local state and a remote side effect. To handle the ambiguous interval between applying an effect and recording confirmation, the prototype tests an idempotent exchange and a reconciliation-first recovery strategy. The guarantees therefore depend on the exchange's idempotency contract or a reliable reconciliation procedure.

Algorithm 1 Authorization and dispatch path

Require: Wire envelope w, proposal p, time t
Ensure: Either reject or one authorized dispatch attempt

$e \leftarrow \textsc{StrictParse}(w)$
$m, \sigma \leftarrow \textsc{Extract}(e)$
$K \leftarrow \textsc{TrustedKeyLookup}(m.keyid)$
if K is missing or key status is invalid at t **then**
    return reject
end if
if $e.alg \neq K.configuredAlg$ **then**
    return reject
end if
if $\mathsf{Verify}_K(D \| C(e \setminus \{\sigma\}), \sigma) = 0$ **then**
    return reject
end if
if $\textsc{MandateContextValid}(m, t) = 0$ **then**
    return reject
end if
if $\mathsf{H}(C(p)) \neq m.proposalHash$ **then**
    return reject
end if
if $\textsc{PolicyAllows}(p, m) = 0$ **then**
    return reject
end if
$r \leftarrow \textsc{ReserveDurably}(m.requestId, \mathsf{H}(C(p)))$
if $r \neq$ NEW **then**
    return reject
end if
$\textsc{SetState}(m.requestId,$ DISPATCHING$)$
$x \leftarrow \textsc{SubmitIdempotently}(m.requestId, p)$
if x is confirmed **then**
    $\textsc{SetState}(m.requestId,$ CONFIRMED$)$
    return accept
else
    $\textsc{ReconcileOrDefer}(m.requestId)$
    return PENDING
end if

E. Effect handling and recovery

The simulated exchange stores effects and, when configured as idempotent, stores an idempotency key in the same transaction as the effect. Repeated submissions with the same key do not create another effect. The crash harness injects failures after reservation, after marking dispatch, after applying the effect, after confirmation, and after a timeout that occurs after application. It then recovers and retries according to the selected strategy.

The key distinction is between *request uniqueness* and *effect uniqueness*. Request uniqueness is enforced by the ledger. Effect uniqueness requires either an idempotent receiver, a transactional boundary shared with the effect, or a reconciliation protocol that can determine whether the effect already happened. Without one of these, retrying after an ambiguous timeout can duplicate the effect.

F. Design ablations

The experiment uses nine named configurations: `D0 none`, `D1 policy`, `D2 +hmac`, `D3 +expiry`, `D4 +mem replay`, `D5 +durable`, `D6 +life+bind`, `D7 full`, and `D7n naive alg`. These are cumulative engineering configurations rather than independent cryptographic primitives. The sequence is useful for illustrating where the tested controls block the constructed attacks, but a causal interpretation of each difference requires care because some configurations change multiple properties.

V. Experimental Methodology

A. Implementation and environment

The prototype is implemented in Python and uses SQLite for the registry, ledger, and simulated exchange. The recorded full run reports Python 3.13.15, Linux kernel 6.18.48+, NumPy 2.1.3, pandas 2.3.3, cryptography 50.0.1, SQLite 3.45.1, and four available CPUs. The workload generator uses seed 1337 for repeatable proposal construction. Cryptographic test keys are freshly generated per run and are not derived from the public seed.

The full attack matrix uses 60 cases per family. It evaluates 19 families in total: one valid control, 17 attack families, and one residual-risk family. For each of the nine designs, this produces 1,140 requests including the control and residual family. The attack-only denominator is 17 attack families times 60 requests, or 1,020 requests per design. The notebook reports executed requests, execution rate, a 95% Wilson interval, and the most common gate result for each family. These intervals describe the binomial rate under the generated workload; they do not cover model misspecification or attack-family selection bias.

The family set includes unsigned forgery, signature bit flips, amount tampering, approval tampering, proposal substitution, exact replay, replay after restart, unknown keys, revoked keys, expired and future keys, expired mandates, audience mismatch, policy-version downgrade, signed over-limit requests, verifier-material leakage, malformed messages, and in-policy semantic manipulation. The last family is classified as residual risk rather than as an attack the gate is expected to reject.

B. Crash and recovery experiment

The crash harness injects failures at five boundaries: after reservation, after dispatch marking, after effect application, after confirmation, and after a timeout following effect application. It compares non-idempotent and idempotent exchanges and a reconciliation-first recovery option. Each recorded crash configuration uses ten repetitions in the full run. The outcome is classified according to the number of effects observed after recovery and retry.

C. Concurrent race experiment

For each trial, eight worker processes submit the same signed request concurrently through a process barrier. The test compares `D7 full`, which uses a durable ledger, with `D4 +mem replay`, which uses process-local replay tracking.

TABLE I
AGGREGATE EXECUTED ATTACK REQUESTS IN THE RECORDED FULL RUN. LOWER IS BETTER.

Design	Effects	Requests	Rate
D0 none	990	1020	97.1%
D1 policy	900	1020	88.2%
D2 +hmac	600	1020	58.8%
D3 +expiry	540	1020	52.9%
D4 +mem replay	480	1020	47.1%
D5 +durable	420	1020	41.2%
D6 +life+bind	60	1020	5.9%
D7 full	0	1020	0.0%
D7n naive alg	60	1020	5.9%

Thirty trials are performed for each configuration. The primary outcome is the number of recorded exchange effects per trial; worker return reasons and internal errors are also recorded.

D. Revocation and performance

The revocation experiment warms a key cache, revokes the key, and then submits requests at increasing times after revocation. Cache time-to-live settings of 0, 5, and 60 seconds are compared. End-to-end latency tests compare a no-check baseline, a legacy-storage HMAC configuration, Ed25519 with DELETE/FULL, Ed25519 with WAL/FULL, Ed25519 with WAL/NORMAL, and WAL/FULL with a five-second key cache. Each configuration uses five repetitions of 2,000 measured requests after 30 warm-up requests. The notebook reports median, percentile latency, throughput, and a bootstrap interval for the pooled median. These measurements are local prototype timings, not estimates of production service capacity.

VI. RESULTS

A. Attack matrix

Table I summarizes the aggregate number of attack requests that caused an effect. Each design is exposed to the same 1,020 generated attack requests. The full design records zero attack effects in this set while accepting all 60 valid control requests. The residual in-policy manipulation family is deliberately excluded from the attack-only denominator and executes under every design.

The aggregate trend is consistent with the intended layered design: policy checks alone do not authenticate the caller; signatures do not by themselves enforce expiry or replay resistance; and durable replay protection does not replace key lifecycle and proposal binding. The full design blocks all constructed attack requests in the tested matrix. This result is bounded by the finite set of attack families and the implementation's own assumptions. It should not be read as a general false-acceptance rate for real-world attacks.

The naive algorithm-selection design executes 60 requests from the verifier-material leakage family. This is an expected failure of the intentionally weakened configuration in the test harness. It illustrates why the accepted algorithm must be pinned to trusted configuration and why shared-secret and public-key systems have different compromise consequences.

TABLE II
ILLUSTRATIVE CRASH OUTCOMES FROM THE RECORDED HARNESS.

Exchange/recovery	Failure point	Verdict
Non-idempotent, blind retry	after reserve	exactly-once
Non-idempotent, blind retry	after dispatch mark	exactly-once
Non-idempotent, blind retry	after apply	duplicate
Non-idempotent, blind retry	timeout after apply	duplicate
Non-idempotent, reconcile-first	timeout after apply	exactly-once
Idempotent exchange	timeout after apply	exactly-once

TABLE III
CONCURRENT DUPLICATE-SUBMISSION EXPERIMENT.

Design	Trials	Workers	Effects/trial	Internal errors
D4 +mem replay	30	8	8 (all trials)	0
D7 full	30	8	1 (all trials)	0

B. Valid controls and semantic residual risk

All nine designs accept the 60 valid control requests in the recorded matrix. The in-policy semantic manipulation family also executes under every design. This is not a contradiction: cryptographic authorization can establish that a request was signed and bound to a proposal, but it cannot determine whether the authorized goal is sensible, aligned with user intent, or based on trustworthy upstream information.

This residual risk matters for AI systems. A robust architecture should separate deterministic authorization from semantic risk assessment and should treat policy design, approval quality, and provenance of the proposal as independent security concerns. The experiment demonstrates the boundary of the gate rather than solving semantic safety.

C. Crash recovery

The crash experiment shows that a local ledger is not sufficient to guarantee one external effect in every case. With a non-idempotent exchange and blind retry, a crash after application or a timeout after application can produce duplicate effects. In the recorded harness, idempotent exchange behavior or reconciliation-first recovery avoids this duplication across the tested crash points.

The word "exactly-once" in Table II is the harness's outcome label: one effect was observed for the modeled execution and recovery sequence. It is not a claim that arbitrary remote systems can provide exactly-once semantics without a suitable contract. In deployment, the external service must support a stable idempotency key or expose enough state for reliable reconciliation.

D. Concurrent submissions

The race experiment gives a clear contrast between local and durable replay state. For `D4 +mem replay`, all 30 trials recorded eight effects, one from each worker. For `D7 full`, all 30 trials recorded exactly one effect, with no internal errors in either configuration.

The result supports the claim that the tested durable reservation and idempotent exchange prevent duplicate effects under

TABLE IV
RECORDED END-TO-END BENCHMARK RESULTS.

Configuration	Median (ms)	p99 (ms)	Req./s
No-check baseline	0.008	0.032	81322
HMAC, legacy storage	19.640	26.775	48.9
Ed25519, DELETE/FULL	18.325	25.560	52.6
Ed25519, WAL/FULL	4.393	10.420	201.1
Ed25519, WAL/NORMAL	0.320	0.618	2482.5
WAL/FULL, 5 s cache	4.508	12.653	194.9

this local multi-process workload. It does not test multiple hosts, storage replication, filesystem corruption, or every possible process-kill timing. Those are important next steps for a broader distributed-systems evaluation.

E. Revocation staleness

With a zero-second cache TTL, all requests after revocation are rejected in the recorded experiment. With a five-second TTL, a request one second after revocation is accepted from the warmed cache, while later requests are rejected. With a 60-second TTL, requests one and nine seconds after revocation are accepted, while the request 119 seconds after revocation is rejected. This is consistent with a bounded stale-acceptance window controlled by cache TTL, subject to the test's timing and cache semantics.

The result exposes an operational trade-off. Caching can reduce repeated registry reads, but revocation responsiveness is then limited by cache invalidation or expiry. High-risk deployments may need push-based invalidation, short TTLs, or an online status check for sensitive actions.

F. End-to-end performance

Table IV reports the recorded benchmark output. The no-check baseline is not a security-equivalent comparator; it is included only to show the cost of the complete path relative to a minimal submission path. The full Ed25519 WAL/FULL configuration reports a median latency of 4.393 ms and mean throughput of 201.1 requests/s. WAL/NORMAL reports a lower median latency of 0.320 ms and throughput of 2,482.5 requests/s, at the cost of weaker durability guarantees than FULL synchronization under some failure scenarios. The paper should not recommend a synchronization mode without considering the application's data-loss and durability requirements.

These figures are environment-specific. The large difference between storage configurations suggests that database journaling and synchronization dominate the measured end-to-end path in this local setup. It would be incorrect to attribute the full difference to signature verification alone. The benchmark is also sequential and does not establish throughput under realistic network, queueing, or multi-host load.

VII. FORMALIZATION OF THE AUTHORIZATION BOUNDARY

A. Entities and message fields

For clarity, we distinguish five entities. The *proposer* creates a candidate action; it may be a language model, a deterministic planner, or an application component. The *authorizer* decides whether the candidate action may be attempted and signs a mandate. The *key registry* maps a key identifier to trusted verification material, an algorithm, and lifecycle metadata. The *gate* validates the mandate and coordinates execution. Finally, the *exchange* represents the downstream service that applies the action.

The prototype uses a deliberately small action vocabulary: a token, a BUY or SELL action, and an integer amount in cents. This schema is not intended to model a complete trading system. It makes authorization and effect semantics precise enough to test. The amount is represented as an integer rather than a floating-point value so that serialization and equality checks do not depend on rounding behavior. The wire parser rejects floating-point values and non-finite constants, and it enforces a maximum message size of 8,192 bytes. Request identifiers are restricted to a bounded character set and length, and token names follow a separate allowlist-shaped expression.

A mandate contains a request identifier, audience, policy version, issue time, expiry time, proposal digest, and a decision object. The decision object repeats the action fields to make the authorization decision explicit. This duplication is intentional: the gate compares the decision fields to the proposal rather than assuming that the presence of a valid signature automatically binds every application-level field correctly. The proposal digest is computed over the canonical proposal representation and compared with a constant-time digest comparison routine.

B. Acceptance predicate

Let w denote the received wire message, p the supplied proposal, t the evaluation time, K the resolved key record, and L the durable ledger. The full design accepts a request only if the following predicate holds:

$$\begin{aligned}\mathsf{Accept}(w,p,t) \equiv\ & \mathsf{ParseStrict}(w) \wedge \mathsf{SchemaValid}(w) \\ & \wedge \mathsf{KeyKnown}(K) \wedge \mathsf{KeyActive}(K,t) \\ & \wedge \mathsf{AlgorithmPinned}(w,K) \wedge \mathsf{SignatureValid}(w,K) \\ & \wedge \mathsf{TimeValid}(w,t) \wedge \mathsf{ContextValid}(w) \\ & \wedge \mathsf{ProposalBound}(w,p) \wedge \mathsf{PolicyAllows}(p) \\ & \wedge \mathsf{ReservationNew}(L,w). \end{aligned} \tag{8}$$

The ordering in the implementation is security-relevant. Parsing and schema checks occur before nested fields are used. Signature and key checks happen before a mandate is treated as trusted. Context and proposal checks prevent a valid signed object from being used in the wrong service or with a different action. The durable reservation is the serialization point for competing workers. The exchange's idempotency behavior is a separate property and is not implied by Eq. (8) alone.

C. Safety properties and their scope

We use the following properties to interpret the experiments. They are engineering properties of the implemented state machine under stated assumptions, not cryptographic proofs of a deployed system.

P1: Invalid-message non-execution.
A request rejected by parsing, authentication, context, binding, or policy validation must not create an exchange effect.

P2: Single durable reservation.
For a given request identifier and fingerprint, at most one concurrent process obtains the new-reservation result from the SQLite ledger.

P3: Request conflict detection.
Reusing a request identifier with a different fingerprint is not treated as a valid retry.

P4: Effect uniqueness under idempotency.
When the exchange atomically records an idempotency key with the effect, repeated submissions using that key create at most one stored effect in the simulator.

P5: Recovery progress.
Under the modeled recovery strategy, a request left in an intermediate state can be reconciled or retried without introducing a duplicate effect in the safe configurations tested.

P1 is evaluated by the attack matrix. P2 is examined by the multi-process race test. P3 is examined by request identifier conflict tests. P4 and P5 are examined by the crash harness and the finite-state model checker. The tests do not prove these properties for arbitrary database failures, compromised hosts, or external APIs. In particular, the model's state space is only as faithful as its transitions and abstraction boundaries.

P1 is evaluated by the attack matrix. P2 is examined by the multi-process race test. P3 is evaluated through conflicting-identifier requests, while P4 is examined by the crash harness and finite-state model checker. The tests do not prove these properties for arbitrary database failures, compromised hosts, or external APIs. In particular, the model's state space is only as faithful as its transitions and abstraction boundaries.

VIII. Implementation Details

A. Strict parsing and schema validation

The parser uses a duplicate-key-rejecting object hook. This avoids accepting a message whose meaning changes depending on whether a parser chooses the first or last occurrence of a repeated JSON member. It also rejects floats and non-finite numeric constants at parse time, rather than allowing them to flow into policy logic. After parsing, schema validation checks the protocol version, algorithm label, key identifier, signature encoding, mandate fields, action values, amount type, timestamps, and request identifier format.

Strict parsing is not a substitute for cryptographic verification. It removes ambiguity from the representation before the verifier acts on it. Conversely, a valid signature over an ambiguous or differently interpreted object can still leave application components disagreeing about what was approved. The parser and signer therefore share one canonicalization routine in the prototype. A deployment using different programming languages should include positive and negative conformance vectors, especially for Unicode, integer boundaries, duplicate keys, escaped characters, and nested objects.

B. Signature input and algorithm selection

The signed input begins with the fixed byte string `AgentShield-v1\0`. The prefix provides domain separation: a signature intended for this protocol should not accidentally be accepted as a signature for an unrelated protocol that happens to serialize the same fields. The canonicalized input covers the version, algorithm identifier, key identifier, and mandate. The signature field itself is excluded from the signed body.

Algorithm selection is not delegated to an untrusted sender. In the full design, the registry record determines the expected algorithm, and the envelope label must match that configured algorithm. The intentionally weakened variant uses the envelope label to select verification behavior. That change makes the verifier-leak experiment meaningful: a shared HMAC secret can generate a valid MAC, whereas reading an Ed25519 public key does not provide the private signing key. The test is therefore an illustration of key-type semantics and algorithm pinning, not a claim that public-key verification is immune to private-key compromise.

C. Time validation and key lifecycle

The prototype sets a maximum mandate lifetime of 600 seconds and a clock-skew allowance of five seconds. It requires the issue time not to be too far in the future, the current evaluation time to precede the expiry time, and the interval between issue and expiry to remain within the maximum. Key records separately represent lifecycle states, including active, revoked, expired, and not-yet-valid cases. This separation matters because a mandate may be within its own validity window while the signing key is no longer trusted.

The evaluation uses a fixed evaluation time for the attack matrix so that the same generated request has deterministic temporal semantics across designs. The revocation experiment is different: it warms the cache, revokes the key, and submits requests at specified offsets to reveal the cache's stale window. These are complementary tests. Fixed time supports controlled comparison; timed revocation probes the behavior of cache freshness.

D. Policy and proposal binding

The policy component checks the token allowlist, action vocabulary, and maximum amount. Policy versioning prevents a mandate issued under an older policy from silently bypassing a newer minimum policy version. The audience field prevents a mandate intended for one service from being replayed at another service that shares key material but has a different role. The proposal binding check compares the decision's action fields to the received proposal and recomputes the proposal digest.

These checks address different errors. Policy limits the permitted action space. Audience prevents cross-service use. Versioning prevents stale policy context. Binding ensures that the specific action submitted to the exchange is the action that the mandate authorized. None of these checks can infer whether an allowed action matches the user's underlying intent. That question belongs to the quality of the upstream authorization decision and the policy-governance process.

E. Ledger schema and transactional boundary

The ledger uses SQLite transactions and a uniqueness constraint over the request identifier. Reservation is performed inside a write transaction using an immediate transaction boundary, so competing writers serialize before the insert decision. The row records a request fingerprint and state. An existing identifier with the same fingerprint is treated as a duplicate; an identifier paired with a different fingerprint is treated as a conflict. This distinction prevents an attacker from using a previously accepted identifier as a vehicle for a different proposal.

The exchange has its own effect table and, in idempotent mode, an idempotency table. The simulator inserts the idempotency key and effect within one database transaction. This arrangement makes the simulated exchange's deduplication atomic. It should not be conflated with a real external API unless that API explicitly provides an equivalent durable idempotency contract. If the external service neither accepts idempotency keys nor supports reliable status queries, a client cannot infer with certainty whether an operation occurred merely from a timeout.

IX. Detailed Experimental Protocol

A. Research questions

The evaluation asks five questions: how attack execution changes across ablations; whether valid controls remain accepted; which recovery configurations avoid duplicate effects; whether durable state prevents duplicates under simultaneous submissions; and how cache TTL and SQLite settings affect revocation and latency. These questions are reported separately because a single aggregate rate cannot explain replay behavior, recovery correctness, or performance trade-offs.

B. Attack-family construction

The test corpus contains 19 named families: one valid control, 17 protocol-level attack families, and one residual-risk family. The protocol attack families are unsigned forgery, signature bit flip, amount tampering, approval tampering, proposal substitution, exact replay, replay after restart, unknown key, revoked key, expired key, future key, expired mandate, audience mismatch, policy downgrade, signed over-limit action, verifier-material leakage, and malformed wire data. The residual-risk family is an in-policy semantic manipulation.

Each family uses 60 generated cases in the recorded full run. The proposal generator chooses from BTC, ETH, and SOL, selects BUY or SELL, and chooses an integer amount in the configured range for ordinary proposals. The over-limit family deliberately exceeds the policy maximum. The signed and malformed variants are constructed to isolate specific protocol failures. Replay tests first execute a request and then submit it again; the restart variant reopens the relevant gate state so that persistence is tested rather than only a process-local set.

The construction has a clear strength and a clear limitation. Its strength is repeatability: each family targets an interpretable property and is applied consistently across designs. Its limitation is that the family generator is authored alongside the implementation. A stronger independent evaluation would use a separately implemented attack generator, property-based fuzzing, mutation testing, and external reviewers who do not know the expected result for each case.

C. Denominators and interpretation

The attack-only denominator is 17 families multiplied by 60 cases, yielding 1,020 attack requests per design. The valid control has 60 requests per design and the residual family has 60 additional requests per design. The attack-only denominator excludes both controls and residual-risk cases; this distinction is important when interpreting the zero-effect result. The full design's 0/1,020 observed attack execution rate is a descriptive result for this generated corpus. It is not an estimate that the probability of a real-world attack succeeding is zero.

For a binomial proportion, the notebook calculates a 95% Wilson score interval. Even when no successes are observed, the interval has a non-zero upper bound. More importantly, the interval assumes a meaningful sampling model. Since the test cases are generated from designed families rather than sampled from an operational attack distribution, the interval describes sampling uncertainty within the chosen test construction and does not account for missing attack classes or model-selection bias. We therefore report counts and rates as primary evidence and treat the confidence interval as a supplementary descriptive statistic.

D. Ablation design and interpretation

The ablations are cumulative. D0 is a minimal implementation that loosely parses the message and trusts the approval field. D1 adds strict parsing and policy checks. D2 adds HMAC authentication. D3 adds mandate expiry. D4 adds process-local replay tracking. D5 changes replay protection to durable storage. D6 adds key lifecycle and proposal/context binding. D7 uses Ed25519 and an idempotent exchange. D7n is the deliberately weakened algorithm-selection variant.

This progression is intended to expose failure classes, not to assert that each step is an isolated causal treatment. Some steps change more than one feature or change the interaction between features. The observed reduction from one row to the next should therefore be described as an ablation trajectory. A future study should add factorial or pairwise feature ablations, where feasible, to estimate interaction effects more rigorously. In particular, cryptographic scheme, algorithm pinning, durable replay state, and downstream idempotency should be varied independently where the design permits.

E. Crash-point protocol

The harness injects abrupt process termination after reservation, after the dispatch state is marked, after the exchange applies an effect, after confirmation is written, and at a timeout immediately following application. The harness then starts recovery and may retry the request. Four configurations compare a non-idempotent exchange with blind retry, a non-idempotent exchange with reconciliation-first behavior, an idempotent exchange with blind retry, and an idempotent

exchange with reconciliation-first behavior. Ten repetitions are used per crash configuration in the recorded full run.

A crash point is not equivalent to a general distributed failure. Process termination tests whether persisted state survives the selected boundary, but it does not reproduce every disk-controller behavior, torn write, network partition, delayed remote response, or database corruption. The results are strongest when stated as observations about these explicit injection points. They motivate an end-to-end requirement: the recovery design must be aligned with the guarantees provided by the effect receiver.

X. Threat-to-Control Mapping

Table V maps each tested attack family to the principal control expected to reject it. This mapping is conceptual: some requests can be rejected by more than one check, and the exact rejection reason depends on validation order. The table describes the intended first-line control rather than claiming that each attack has only one possible defense.

The mapping highlights a design principle: authorization is a conjunction of checks with distinct responsibilities. A signature is not a substitute for policy, and policy is not a substitute for authentication. Similarly, a durable request identifier is not a substitute for idempotent effect handling. Security testing should therefore measure not only the aggregate attack rate but also whether each class is stopped for the expected reason and whether a rejection can leave partial state behind.

XI. Reproducibility Checklist

Before submission, restart the runtime and execute every cell with quick mode disabled. Verify that the manifest records the expected workload sizes, environment, and complete claim list; regenerate all tables and figures from the current CSV files; and preserve the attack-matrix, crash, model-checker, race, revocation, performance, agent-defense, and claims outputs. The race artifact should include worker results, exit codes, final ledger states, and effect totals. Repeat the experiments with at least one additional seed, inspect unexpected outcomes rather than silently excluding them, and pin the dependencies and SQLite settings. Publish only synthetic keys and data. Any code changes after the reported run should trigger regeneration of affected results. This is a recommended checklist, not a claim that every item has already been completed.

XII. Discussion

A. Layered controls are complementary

The attack matrix suggests that no single check addresses all of the tested failure modes. Policy validation constrains the action space but cannot authenticate an authorization. A signature protects integrity but does not establish that a request is current, intended for this service, or not previously consumed. Expiry reduces the lifetime of a mandate but does not stop concurrent duplicates inside the validity window. Durable replay tracking addresses request reuse, while proposal binding prevents a valid mandate from being paired with a different proposal.

The engineering implication is to treat these controls as separate predicates in a gate, with explicit rejection reasons and tests for each predicate. The ablation sequence is useful for demonstrating this separation, although future work should add factorial experiments to isolate interactions more rigorously.

B. A signed decision is not a semantic guarantee

The residual-risk family highlights an important limitation for agentic systems. A mandate can be authentic and structurally valid while approving a poor decision. If the upstream authorizer has misunderstood a request, been manipulated by untrusted context, or applied an incomplete policy, cryptography will preserve the wrong decision faithfully. Security architecture must therefore combine authorization integrity with trustworthy intent capture, policy governance, provenance, risk checks, and meaningful user confirmation where appropriate.

C. Exactly-once is an end-to-end property

The crash tests demonstrate that the local database cannot, by itself, atomically commit an independent external side effect. A local state machine may know that dispatch was attempted without knowing whether the remote operation succeeded. The safe options depend on the external system: use an idempotency key that the receiver durably enforces, use a shared transaction when genuinely available, or reconcile remote state before retrying. The manuscript deliberately limits its claim to the simulated exchange and modeled recovery strategies.

D. Operational trade-offs

The revocation experiment and benchmark show that security settings affect availability and latency. A longer key-cache TTL can improve efficiency while extending the period in which a recently revoked key may remain accepted. WAL/FULL and WAL/NORMAL show different throughput and latency profiles, but durability is part of the threat model rather than a tuning detail. A deployment should document its crash-consistency requirements and measure performance on its actual storage and workload.

XIII. Limitations

Several limitations must be addressed before treating the prototype as evidence of deployment readiness.

First, the attack corpus is hand-constructed. It covers useful protocol-level classes but is not sampled from a representative population of real attacks. A zero count in this corpus does not imply a zero real-world attack probability. Second, the system is a local Python/SQLite prototype with a simulated exchange. It does not evaluate network partitions, remote service behavior, replicated ledgers, hardware-backed keys, production identity systems, or adversarial control of the host. Third, the key registry and policy are assumed trusted. Fourth, the agent experiment uses scripted scenarios and does not provide a statistically grounded study across language models, prompts, tasks, or tool ecosystems. Fifth, performance measurements are environment-specific and should be repeated on a documented target deployment. Sixth, the current ablation sequence is

TABLE V
TESTED ATTACK FAMILIES AND THEIR PRIMARY INTENDED CONTROLS.

Attack family	Primary control	Rationale
Unsigned forgery	Signature verification	An unsigned or empty-signature envelope must not authenticate a mandate.
Signature bit flip	Signature verification	Changing a signed byte should invalidate verification.
Amount tampering	Signature and proposal binding	A modified amount must not inherit the original approval.
Approval tampering	Signature verification	The approval bit is part of the signed mandate and cannot be toggled after signing.
Proposal substitution	Proposal digest and field equality	The action sent to the exchange must match the approved action.
Exact replay	Durable request ledger	A previously consumed request identifier must not reserve a second dispatch.
Replay after restart	Persistent ledger	Replay protection must survive a new process and a newly opened gate.
Unknown key	Trusted key registry	A key identifier not present in trusted configuration is rejected.
Revoked or expired key	Key lifecycle check	A signature from a key that is no longer active is not accepted.
Future key	Key validity interval	A key not yet valid cannot authorize an action.
Expired mandate	Mandate lifetime check	An old authorization must not remain usable indefinitely.
Audience mismatch	Audience binding	A mandate for another service is rejected.
Policy downgrade	Minimum policy version	A stale policy version cannot override the current policy requirement.
Signed over-limit action	Deterministic policy enforcement	Authenticity does not permit an action outside the configured limit.
Verifier-material leakage	Algorithm pinning and key-type separation	A sender cannot choose an alternate verification mode merely by changing the envelope label.
Malformed wire	Strict parser and schema validation	Ambiguous or truncated encodings are rejected before dispatch.
In-policy semantic manipulation	Not blocked by protocol controls alone	A valid, authorized action may still be undesirable if the upstream decision or policy is wrong.

cumulative and therefore does not fully isolate all feature interactions. Finally, the current manuscript's related-work analysis must be completed and verified against the final venue requirements before submission.

These limitations do not make the experiments useless. They define what the results can support and identify the next evaluation steps: independently generated attacks, broader concurrency and failure schedules, multiple process and host configurations, explicit threat-model review, and a comparison with relevant authorization and agent-security systems.

XIV. CONCLUSION

This paper presented AGENTSHIELD-CRYPTO, a prototype for evaluating cryptographic authorization and effect handling in a tool-using AI-agent workflow. In the recorded synthetic evaluation, the full design blocked all 1,020 constructed attack requests while accepting all 60 valid controls. Across 30 trials with eight concurrent workers, the durable-ledger design recorded one effect per trial, compared with eight effects per trial under process-local replay tracking. The crash experiments showed that duplicate prevention after an ambiguous failure depends on idempotent effect handling or reliable reconciliation. Revocation tests exposed the expected trade-off between cache efficiency and stale acceptance, while local benchmarks showed that SQLite settings materially affect end-to-end latency.

These findings do not mean that cryptography makes an agent safe. Rather, tool execution requires separate controls for authorization integrity, policy, context, proposal binding, persistent replay protection, and effect recovery. None can establish that an in-policy action reflects the user's actual intent. Future work should use independently generated attacks, broaden distributed failure schedules, validate the model against implementation traces, and compare the prototype with existing agent-security systems. The current evidence is limited to the stated synthetic workloads and local simulator; production readiness and universal exactly-once behavior are not claimed.

ACKNOWLEDGMENT

No external funding or institutional support is claimed in this draft. Add acknowledgments only where factually appropriate.

REFERENCES

[1] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," RFC 8032, Jan. 2017, doi: 10.17487/RFC8032.
[2] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, Feb. 1997, doi: 10.17487/RFC2104.
[3] A. Rundgren, B. Jordan, and S. Erdtman, "JSON Canonicalization Scheme (JCS)," RFC 8785, June 2020, doi: 10.17487/RFC8785.
[4] H. Birkholz *et al.*, "Remote ATtestation procedureS (RATS) Architecture," RFC 9334, Jan. 2023, doi: 10.17487/RFC9334.
[5] L. Lamport, "Paxos Made Simple," *ACM SIGACT News*, vol. 32, no. 4, pp. 18–25, Dec. 2001, doi: 10.1145/568425.568433.
[6] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017.
[7] SQLite Consortium, "Write-Ahead Logging," SQLite Documentation. [Online]. Available: https://www.sqlite.org/wal.html. Accessed: Oct. 15, 2026.
[8] OWASP GenAI Security Project, "LLM Top 10 for Large Language Model Applications," project documentation. [Online]. Available: https://genai.owasp.org/. Accessed: Oct. 15, 2026.